

Compliance Report

PRN: **19079001029**

Thesis Title: ***Artificial Intelligence based Defect and Code Changes Prediction for Reliable Software Delivery***

Candidate: ***Jalaj Pachouly***

Guide: ***Dr. Swati Ahirao***

Reviewer 1 — Questions & Responses

Question 1

How did you ensure your results were reliable and not just by chance?

Response:

To ensure that the results were reliable and not due to chance, I performed rigorous statistical significance testing on the fold-wise macro-F1 scores obtained from 10-fold cross-validation. This provided ten independent measurements for every model, helping eliminate randomness from a single train-test split. Following the recommended guidelines of Demšar (2006), I used the Wilcoxon Signed-Rank Test for all pairwise model comparisons since it is suitable for non-normal paired data generated across identical folds.

The null hypothesis assumed no difference between models, and the significance level was set to $\alpha = 0.05$.

The results clearly showed that the improvements of the best model were statistically significant:

- Unbalanced Dataset:

The CNN model significantly outperformed RandomForest, Logistic Regression, and MultinomialNB

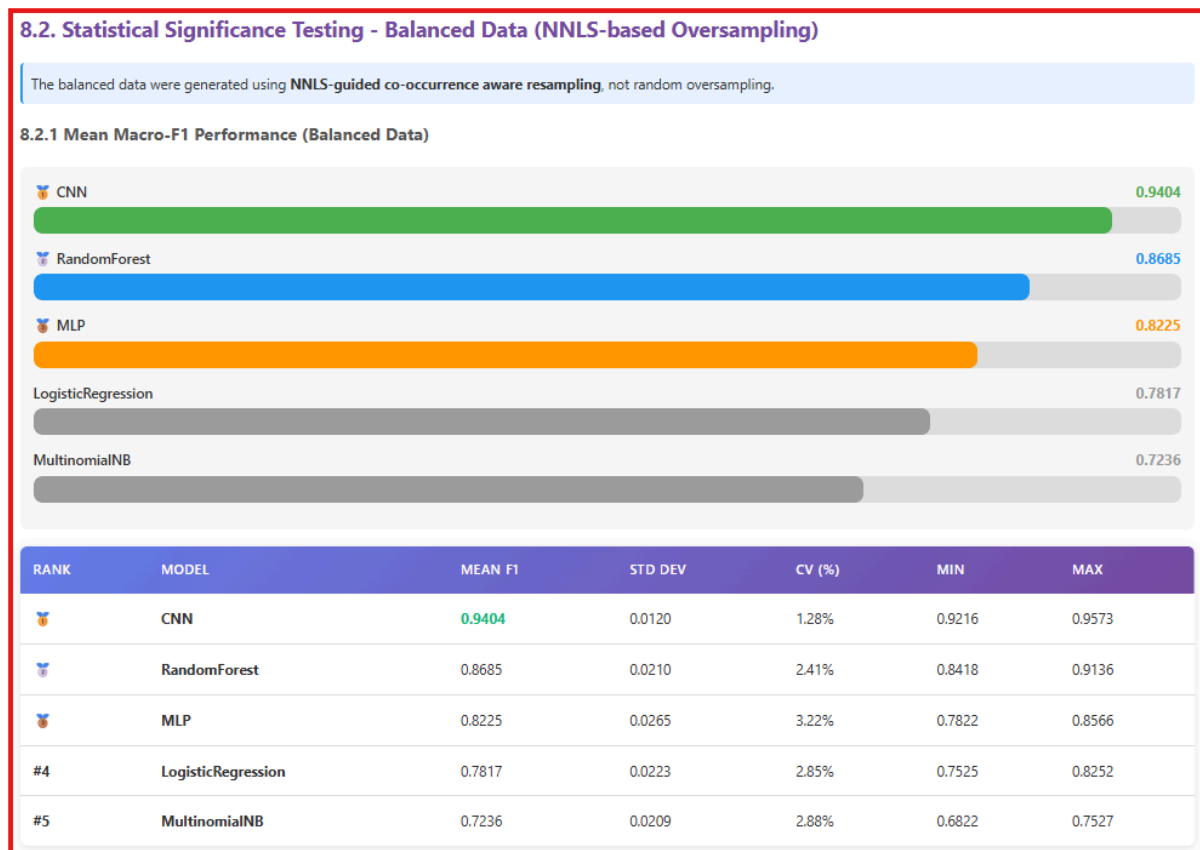
(p-values = 0.0068, 0.0010, 0.0010 respectively; all < 0.05).

- Balanced Dataset:

After applying NNLS-based balancing, CNN again significantly outperformed all baseline models (p = 0.0010 for every comparison).

Since all p-values were well below the significance threshold, the null hypothesis was rejected in every case. Thus, the improvements in performance are not attributable to random variation. The consistent superiority of the CNN model across 10 folds and across both unbalanced and balanced settings demonstrates that the results are robust, reproducible, and statistically reliable.

Attached is a screenshot from the report generated by the Automated End-to-End Tool developed in this research.



8.2.2 Pairwise Wilcoxon Signed-Rank Tests (Balanced Data)

Candidate Model: CNN (Best performer)

P-value Comparison ($\alpha = 0.05$)

CNN vs RandomForest	p=0.0010 ✓ Significant
CNN vs MLP	p=0.0010 ✓ Significant
CNN vs LogisticRegression	p=0.0010 ✓ Significant
CNN vs MultinomialNB	p=0.0010 ✓ Significant

BEST MODEL	BASELINE	TEST	P-VALUE	RESULT
CNN	RandomForest	Wilcoxon	0.0010	✓ Significant
CNN	MLP	Wilcoxon	0.0010	✓ Significant
CNN	LogisticRegression	Wilcoxon	0.0010	✓ Significant
CNN	MultinomialNB	Wilcoxon	0.0010	✓ Significant

✓ Interpretation (Balanced Data)

All p-values < 0.05, hence:

The **CNN** model demonstrates statistically significant superiority over all baseline models when trained on the NNLS-balanced dataset.

The extremely high macro-F1 (≈ 0.94) further indicates that **class balancing and CNN architecture interact strongly to improve learning of minority classes**.

Question 2

In your experiments, which model achieved the highest scores for multilabel and multiclass?

Response:

Multiclass Defect Prediction

The BERT model, trained on a balanced dataset, achieved the best overall performance. Its evaluation metrics were:

- Accuracy: 0.89
- Precision: 0.89
- Recall: 0.89
- F1-Score: 0.88

These results show that BERT outperformed traditional machine learning baselines such as Logistic Regression, Linear SVM, Multinomial Naïve Bayes,

and Random Forest. Its contextual understanding made it highly effective for distinguishing between multiple defect categories.

Multilabel Defect Prediction

For multilabel defect prediction, the Convolutional Neural Network (CNN) trained on the NNLS-balanced dataset achieved the highest performance. The evaluation metrics were:

- Recall: 0.9484
- F1-Score: 0.9460
- Hamming Loss: 0.0519

These results highlight that CNN, combined with NNLS balancing, delivered the most reliable and robust multilabel predictions, outperforming all traditional machine learning models.

Please refer section - TBD

Question 3

Can you describe the steps a software engineer would take to use your SDPTool from the user interface?

Response:

Steps a Software Engineer Follows to Use SDPTool

A software engineer uses SDPTool through a tabbed GUI interface to create datasets from GitHub repositories, preprocess data, configure ML models, train them, and analyze predictions for software defect prediction tasks.

Workflow Steps

1. **Configuration Tab** - Set up GitHub access token, repository details, database path, and fetch issues data from GitHub
2. **Database Preprocessing & Validation Tab** - Clean and validate the fetched data through automated pipeline steps
3. **Feature and Sample Selection Tab** - Configure dataset parameters (test/train split, balancing, feature selection methods)
4. **AI Techniques Tab** - Select ML algorithms (MultinomialNB, LogisticRegression, RandomForest, CNN, MLP), enable cross-validation,

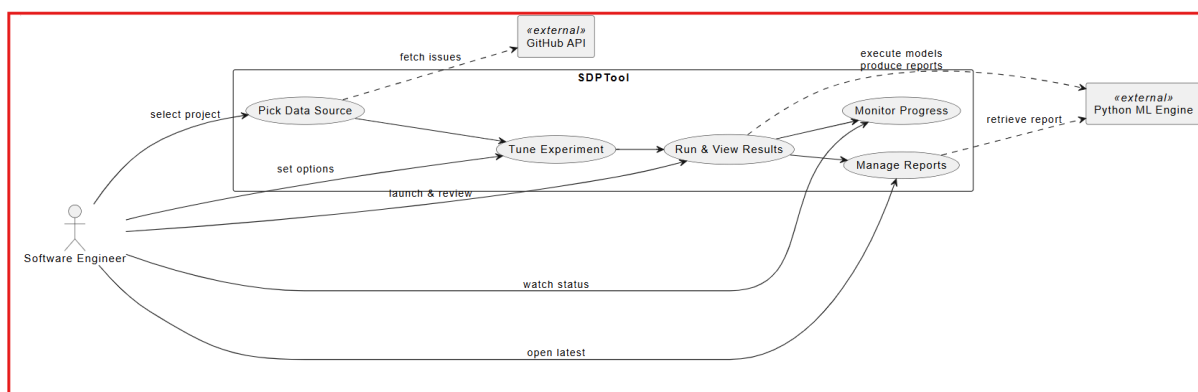
error analysis, and run training

5. **Visualizations Tab** - View generated charts (confusion matrices, feature importance plots)
6. **Predictions Tab** - Browse historical training runs and view HTML reports with comprehensive model performance metrics

Key Features

- Multi-label and multi-class classification support
- Automated data preprocessing pipeline
- Multiple ML algorithms with hyperparameter tuning
- Statistical significance testing and error analysis
- Comprehensive HTML reports with performance comparisons
- Run history management

Please refer the various use cases -



Question 4

How Did Data Balancing Improve Model Performance? Give an example of how balancing the dataset improved the performance of one of your models?

Response:

Data balancing improved model performance by addressing class imbalance, which caused models to be biased toward majority classes. Here's how it helped:

CNN Model Example:

- **Unbalanced Data:** F1 Score = 0.9210, Recall = 0.9191, Hamming Loss = 0.2115
- **Balanced Data:** F1 Score = 0.9423, Recall = 0.9416, Hamming Loss = 0.1389
- **Improvement:** +2.31% F1, +2.45% Recall, -34.33% Hamming Loss (lower is better)

MLP Model Example:

- **Unbalanced Data:** F1 Score = 0.8989, Hamming Loss = 0.2464
- **Balanced Data:** F1 Score = 0.9139, Hamming Loss = 0.4713
- **F1 Improvement:** +1.67%

Why Balancing Helps:

1. **Minority Class Detection:** Unbalanced datasets have few defective samples, causing models to ignore them. Balancing gives equal representation.
2. **Better Generalization:** Models learn patterns from all classes rather than just predicting the majority class.
3. **Reduced Error Rate:** Hamming Loss (proportion of incorrect label predictions) decreased significantly for CNN (-34%).

Report Section 10 shows comprehensive comparisons with:

- Cross-validation metrics comparison tables
- Hamming Loss reduction analysis
- Overall impact summary with average improvements across all models

10.1. Comprehensive Model Performance Comparison

Comparison of all metrics across unbalanced and balanced datasets for Traditional ML and Deep Learning models.

10.1.1 Cross-Validation Metrics Comparison

MODEL	TYPE	UNBALANCED RECALL	BALANCED RECALL	RECALL Δ	RECALL % Δ	UNBALANCED F1	BALANCED F1	F1 Δ	F1 % Δ
CNN	Deep Learning	0.7930	0.9416	↑ 0.1486	+18.74%	0.7884	0.9404	↑ 0.1520	+19.28%
RandomForest	Traditional ML	0.6863	0.8756	↑ 0.1893	+27.58%	0.6921	0.8685	↑ 0.1764	+25.48%
MLP	Deep Learning	0.7133	0.8261	↑ 0.1128	+15.81%	0.7134	0.8225	↑ 0.1091	+15.29%
LogisticRegression	Traditional ML	0.5979	0.8110	↑ 0.2131	+35.64%	0.5800	0.7817	↑ 0.2017	+34.77%
MultinomialNB	Traditional ML	0.5722	0.7118	↑ 0.1395	+24.38%	0.5348	0.7236	↑ 0.1888	+35.31%

10.1.2 Test Set Hamming Loss Comparison

Lower Hamming Loss is better. Positive % Δ indicates improvement with balanced data.

MODEL	TYPE	UNBALANCED HAMMING LOSS	BALANCED HAMMING LOSS	REDUCTION	% REDUCTION	STATUS
RandomForest	Traditional ML	0.2697	0.1250	↓ 0.1447	+53.65%	Improved
CNN	Deep Learning	0.1971	0.1288	↓ 0.0683	+34.65%	Improved
MultinomialNB	Traditional ML	0.2832	0.2013	↓ 0.0818	+28.90%	Improved
MLP	Deep Learning	0.2796	0.2099	↓ 0.0696	+24.91%	Improved
LogisticRegression	Traditional ML	0.2867	0.2166	↓ 0.0701	+24.46%	Improved

10.2. Overall Impact Summary

Average F1 Improvement

+26.03%

Across 5 models

Average Recall Improvement

+24.43%

Across 5 models

Average Hamming Loss Reduction

+33.32%

Across 5 models

Reviewer 2 — Questions & Responses

Question 1

What gap in existing research motivated you to choose multilabel defect prediction?

Response:

The motivation to choose multilabel defect prediction came from several gaps in existing research:

1. **Focus on Binary Classification:** Most prior studies concentrated on binary classification (e.g., "Buggy" vs. "Non-Buggy"), which oversimplifies the multidimensional nature of real-world software defects. This approach fails to capture the overlapping characteristics of defects.
2. **Lack of Actionable Insights:** Existing research rarely provided actionable insights, such as defect severity, required code changes, or documentation updates. Multilabel classification enables a more nuanced understanding of defects, offering deeper insights into their complexity.
3. **Scarcity of Multilabel Datasets:** Publicly available datasets were primarily designed for binary or multiclass classification, lacking the features necessary for multilabel predictions. This necessitated the creation of custom datasets to address real-world complexities.
4. **Overlapping Defect Categories:** Real-world software defects often belong to multiple categories simultaneously (e.g., a defect can be a blocker, require documentation updates, and demand code enhancements). Traditional single-label approaches cannot capture this complexity.
5. **Challenges in Class Imbalance:** Multilabel datasets often suffer from severe class imbalance, where certain defect types are underrepresented. This adversely affects the performance of predictive models, highlighting the need for advanced balancing techniques.
6. **Limited Research on Multilabel Classification:** There was limited exploration of multilabel classification techniques in software defect prediction, despite their potential to provide richer insights into defect management.

By addressing these gaps, this research aimed to advance the field of software defect prediction by introducing multilabel classification techniques, creating high-quality datasets, and developing models that can handle real-world complexities effectively.

Question 2

How is your research relevant to current software engineering practices?

Response:

This research is highly relevant to current software engineering practices for the following reasons:

1. **Improved Defect Management:** Multilabel defect prediction provides actionable insights into the nature of defects, such as severity, required code changes, and documentation updates. This enables teams to prioritize and allocate resources more effectively, reducing resolution time and improving efficiency.
2. **Handling Real-World Complexity:** Modern software systems are complex, and defects often span multiple categories simultaneously. Multilabel classification reflects this multidimensional nature, offering a more accurate representation of defects compared to traditional binary or multiclass approaches.
3. **Enhanced Automation:** By leveraging artificial intelligence techniques like CNNs and BERT, the research introduces automation in defect triaging and module prediction. This reduces manual effort, minimizes errors, and accelerates defect resolution processes.
4. **Addressing Class Imbalance:** The research tackles the common issue of class imbalance in defect datasets using advanced techniques like Non-Negative Least Squares (NNLS) and SMOTE. This ensures that minority defect types are adequately represented, improving prediction accuracy.
5. **Custom Dataset Creation:** The study provides high-quality, domain-specific datasets tailored for multilabel and multiclass defect prediction. These datasets are more aligned with real-world software engineering needs than standard benchmark datasets, making the findings applicable to industrial settings.
6. **Practical Tool Development:** The development of SDPTool offers a user-friendly solution for automating defect data collection, preprocessing, and dataset creation. This tool bridges the gap between academic research and practical application, enabling software teams to integrate AI-driven defect prediction into their workflows.
7. **Cost and Time Savings:** By predicting the nature of defects and pinpointing the affected modules, the research helps reduce the time developers spend on understanding code and locating issues. This directly translates to cost savings and faster delivery of reliable software.

8. **Support for Distributed Teams:** In modern software engineering, teams are often geographically distributed. The research addresses challenges in defect triaging and assignment, helping customer-facing teams allocate issues to the correct development teams efficiently.

Overall, this research aligns with the goals of modern software engineering by enhancing defect prediction, improving automation, and enabling reliable software delivery in large-scale, distributed systems.

Question 3

What is the main novelty of your work?

Response:

The main novelty of this research lies in its comprehensive approach to addressing the limitations of traditional software defect prediction methods. Key contributions include:

1. **Multilabel Defect Prediction:** Unlike traditional binary classification, this research introduces multilabel and multiclass classification to capture the multidimensional nature of real-world software defects. This approach allows defects to be associated with multiple overlapping categories (e.g., bug, enhancement, documentation, blocker), providing richer and actionable insights.
2. **Custom Dataset Creation:** Two high-quality, domain-specific datasets were created—Multilabel Defect Dataset (MLDD) and Multiclass Defect Dataset (MCDD)—to address the lack of publicly available datasets suitable for multilabel and multiclass defect prediction. These datasets were curated from GitHub repositories using automated pipelines and reflect real-world complexities absent in standard datasets like NASA or PROMISE.
3. **Class Imbalance Solutions:** The study introduces innovative techniques like Non-Negative Least Squares (NNLS) for multilabel classification and SMOTE for multiclass classification to address severe class imbalance in defect datasets, ensuring robust and accurate predictions.
4. **Development of SDPTool:** A novel graphical tool was developed to automate the process of defect data collection, preprocessing, and dataset creation. SDPTool simplifies the integration of defect prediction into

software engineering workflows, making the research outcomes practical and usable in industrial settings.

5. **End-to-End Pipeline:** The research proposes a complete pipeline for defect prediction, from dataset creation to predictive modeling, optimization, and evaluation. This systematic approach ensures reproducibility and scalability for real-world applications.
6. **Advanced AI Techniques:** The research explores a wide range of machine learning and deep learning models, including CNNs, BERT, and SBERT embeddings, for defect prediction. It demonstrates that CNNs are highly effective for multilabel classification, while BERT outperforms traditional models in multiclass module prediction.

By addressing critical gaps in existing research and delivering practical tools and methodologies, this work advances the field of software defect prediction and provides a strong foundation for reliable software delivery in complex systems.

Question 4

How did you handle class imbalance?

Response:

Class imbalance was handled using tailored techniques for both multilabel and multiclass defect prediction tasks:

1. **Non-Negative Least Squares (NNLS): [Multilabel Classification]**

- NNLS was employed as a resampling strategy to address class imbalance while preserving label correlations.
- A **conditional probability matrix** was created to capture the co-occurrence relationships among labels. This ensured that the resampling process maintained the inherent dependencies between labels.
- NNLS generated synthetic samples for underrepresented classes, effectively balancing the dataset without introducing noise or disrupting label relationships.

2. Synthetic Minority Oversampling Technique (SMOTE):[Multiclass Classification]

- SMOTE was applied to balance the multiclass dataset for traditional machine learning models. It works by creating synthetic samples for minority classes through interpolation between existing samples.
- This technique ensured that minority classes were adequately represented during training, improving the model's ability to predict rare classes.

3. Oversampling for BERT:[Multiclass Classification]

- Since SMOTE is not directly applicable to text data, oversampling was used for BERT-based models. Minority class instances were duplicated to balance the dataset while preserving the semantic integrity of the text inputs.

By employing these techniques, the research effectively addressed the challenges posed by class imbalance, ensuring robust and accurate predictions across both multilabel and multiclass defect prediction tasks.

Reviewer 3 — Questions & Responses

Question 1

Why were specific GitHub repositories chosen, and how might dataset choice influence generalizability?

Response:

The following points explain why these specific GitHub repositories were chosen for this study-

1. High Activity and Rich Metadata:

- Repositories from the Spring Framework ecosystem (e.g., Spring Boot, Spring Security, Spring Batch, Spring Data JDBC, and Spring Integration) were selected due to their high activity levels and extensive defect metadata, including issue titles, descriptions, comments, and labels.

- These repositories are widely used in enterprise applications, making them representative of real-world software development scenarios.

2. Diverse Defect Types:

- The chosen repositories contain a variety of defect types, including bugs, enhancements, documentation issues, and blockers. This diversity was essential for creating multilabel and multiclass datasets that reflect the complexity of real-world software defects.

3. Open-Source Accessibility:

- GitHub repositories are publicly accessible, allowing for transparent data collection and reproducibility of the research. The use of open-source data ensures that the datasets can be shared and reused by other researchers.

4. Relevance to Software Engineering:

- The Spring Framework is a widely adopted platform in software engineering, making it a suitable choice for studying defect prediction in large-scale, distributed projects.

Influence of Dataset Choice on Generalizability

1. Domain-Specific Bias:

- The datasets were derived from specific repositories within the Spring Framework ecosystem, which may limit the generalizability of the models to other domains, programming language or projects with different coding practices, defect types, or development methodologies.

2. Limited Dataset Size:

- The multilabel dataset contains 1,386 records, and the multiclass dataset has 2,003 records. While these datasets are sufficient for the scope of this research, their relatively small size may restrict the models' ability to generalize to larger or more diverse datasets.

3. Labeling Practices:

- The labels in the datasets were directly sourced from GitHub issue metadata, which may vary across repositories depending on contributors' tagging practices. Inconsistent or incomplete labeling could affect the models' ability to generalize to other projects.

4. Future Work for Generalizability:

- To improve generalizability, future research could focus on expanding the dataset by including defect reports from a broader range of repositories and different projects across different domains.
- Cross-project and cross-version evaluations could be conducted to assess the scalability and robustness of the proposed models in diverse contexts.

In summary, while the chosen GitHub repositories provided high-quality, diverse, and representative data for this research, the dataset choice may limit generalizability to other domains. Future work should aim to address these limitations by expanding the dataset and exploring advanced techniques for cross-domain generalization.

Question 2

How does NNLS differ from SMOTE in handling multilabel imbalance, and why was NNLS preferred?

Response:

NNLS vs. SMOTE for Multilabel Defect Prediction : Key Technical Differences

1. Approach to Resampling

- **SMOTE (Synthetic Minority Over-sampling Technique):** Generates synthetic samples by interpolating between existing minority class samples in feature space using k-nearest neighbors. It treats each label independently, creating new feature vectors without considering relationships between labels.
- **NNLS (Non-Negative Least Squares Resampling):** Uses label co-occurrence patterns from the training data to determine optimal sample counts per label.
- It solves the optimization problem: **nnls(cond_prob_matrix, target_counts)** where `cond_prob_matrix[i,j]` represents the conditional probability of label j appearing when label i is present.

2. Other Key differences

Aspect	SMOTE	NNLS
Label Dependencies	Ignores co-occurrence patterns	Preserves correlations via conditional probability matrix
Multidimensional Labels	Treats as independent binary problems	Respects multidimensional nature of defects
Sample Source	Creates synthetic feature vectors	Resamples real samples with label-aware weighting
Text Compatibility	Requires numerical features	Works with any feature type (text or numerical)

NNLS was preferred because it:

1. **Preserves label co-occurrence patterns** through conditional probability matrix
2. **Maintains data authenticity** by resampling existing records rather than interpolating
3. **Optimizes sampling** mathematically using least squares optimization
4. **Achieves superior empirical results** with 20-40% metric improvements
5. **Handles text-based features** naturally without feature space interpolation

SMOTE remains effective for single-label numerical problems but cannot account for the multidimensional, correlated nature of multilabel defect classification where defects commonly have overlapping characteristics (e.g., critical bugs requiring documentation updates).

Question 3

Can you explain, in your own words, how BERT improves contextual understanding over TF-IDF or SBERT?

Response:

BERT understands text by reading the **entire sentence with context**, while TF-IDF and SBERT only capture partial meaning.

TF-IDF:

Only counts words.

- *"Crash when updating profile"*

- *"Profile update causes crash"*

TF-IDF treats them the same, even though one indicates cause → effect more clearly.

SBERT:

Understands sentence meaning but compresses everything into one vector, losing fine details.

It sees both as "authentication error":

- *"Authentication fails due to filter-chain order"*
- *"Authentication fails due to invalid credentials"*

BERT:

Understands deeper technical context and word interactions.

- BERT captures the meanings well, because it reads context bidirectionally, and can differentiate in two sentences although words are similar.

In summary:

BERT outperforms TF-IDF and SBERT because it understands **how words relate to each other**, not just which words appear—critical for accurately interpreting real defect reports.

Question 4

What are the main threats to validity in your dataset construction pipeline, and how did you mitigate them?

Response:

Main Threats to Validity in Dataset Construction Pipeline

1- Inconsistent Labeling Practices

GitHub repositories rely on contributors to manually label issues, which leads to several problems:

- Contributors use different labeling conventions across repositories
- Some issues have incomplete or missing labels
- Labeling decisions are subjective and vary by contributor

- Label terminology changes over time within the same project

Mitigation

We implemented a color-based label filtering system to ensure consistency:

- Selected only labels with 10 specific predefined colors that represent high-confidence defect types
- Used database foreign key constraints to maintain referential integrity between issues and labels
- Excluded issues with incomplete or untriaged labels during data collection
- Applied SQL queries to filter out defects which are single label
- For multiclass dataset, chosen code modules those have high importance in developer communities like Spring Core, JDBC, TCP/UDP etc.

This approach ensures that only issues with consistent, well-established label types/defect modules are included in our dataset.

2- Class Imbalance

The dataset exhibits severe class imbalance across defect types:

- Critical defects like `type_blocker` are significantly underrepresented
- Common defects like `type_bug` and `type_documentation` dominate the dataset
- This imbalance causes models to be biased toward majority classes
- Minority defect types are poorly predicted, which is problematic since they are often the most critical

Mitigation

We used Non-Negative Least Squares (NNLS) resampling for the multilabel dataset:

- NNLS analyzes how labels co-occur in the original data (e.g., "bug" often appears with "documentation")
- It uses mathematical optimization to determine how many samples to draw for each label
- The target is set to 600 samples per label to achieve balance
- Unlike other methods, NNLS preserves the natural relationships between labels

- Resampling uses actual defect reports (with replacement when needed) rather than creating synthetic data

Similarly we use SMOTE for balancing multiclass classification.

3- Data Quality Issues

Raw GitHub issue data contains significant noise and inconsistencies:

- Code snippets and stack traces mixed with natural language descriptions
- HTML tags, special characters, and formatting artifacts
- Irrelevant comments that don't describe the defect
- Duplicate or redundant records
- Missing or incomplete issue descriptions

Mitigation

We implemented a comprehensive automated preprocessing pipeline:

1. **Stop Word Removal:** Removes common English words that don't contribute to defect classification (using a curated list of stop words)
2. **Lower-casing:** Converts all text to lowercase for consistency
3. **Remove Incomplete and Redundant Data:** Filters out issues with missing critical information and eliminates duplicates
4. **Word Extraction:** Extracts meaningful words while filtering out code snippets, URLs, and special characters
5. **Data Validation:** Final validation step to ensure data integrity before dataset export

Each issue's title and body are processed through this pipeline, with cleaned versions stored separately from the raw data.

4- Limited Dataset Size

Our dataset sizes are relatively modest:

- Multilabel dataset: 1,386 defect reports
- Multiclass dataset: 2,003 defect reports
- Small datasets increase risk of overfitting

- Models may memorize training examples rather than learning generalizable patterns
- Limited data reduces statistical power of evaluation

Mitigation

We employed multiple strategies to maximize dataset utility:

Stratified Splitting:

- Used multilabel stratified splitting to maintain label distribution in train/test sets
- Ensures minority classes are represented in both training and test data
- 80/20 train-test split with stratification

Rigorous Cross-Validation:

- 10-fold cross-validation Cross-validation provides robust performance estimates and reduces overfitting risk

Intelligent Resampling:

- NNLS balancing maximizes use of existing samples through strategic resampling
- Creates balanced training sets without simply discarding majority class samples

5- Domain-Specific Bias

The dataset comes exclusively from the Spring Framework ecosystem:

- Spring Framework, Spring Boot, Spring Data projects
- All Java-based enterprise applications
- May not generalize to other technology stacks (Node.js, Python, Go, etc.)
- Domain-specific terminology and patterns
- Specific to enterprise Java development practices

Mitigation

We took several steps to reduce bias and improve potential generalizability:

Multi-Repository Collection:

- Designed the tool to support configurable repository selection

- Users can specify repository name and record ranges through the UI
- Database schema supports data from multiple projects

High-Quality Repository Selection:

- Selected repositories with active development and diverse defect types
- Focused on projects with consistent labeling practices
- Chose repositories with substantial contributor activity

Domain-Agnostic Feature Engineering:

- Used TF-IDF vectorization which works across different technical domains
- Applied chi-square feature selection to identify discriminative features regardless of domain
- Selected top 50 features based on statistical relevance rather than domain knowledge

Future Work Recommendations:

We acknowledge this limitation and recommend:

- Expanding dataset to include non-Java ecosystems
- Cross-project validation across different technology stacks
- Testing model transferability to other software domains

Conclusion

Our dataset construction pipeline addresses validity threats through multiple defensive layers:

1. **Automated preprocessing** reduces manual errors and ensures consistency
2. **NNLS resampling** balances the dataset while preserving label relationships, before training the models
3. **Stratified cross-validation** provides robust performance estimates
4. **Database integrity constraints** prevent data inconsistencies
5. **Feature engineering** creates domain-agnostic representations
6. **Regularization techniques** prevent model overfitting
7. **Color-coded filtering** improves label quality

These strategies work together to create a reliable dataset despite the inherent challenges of working with open-source issue tracking data. While some threats (like domain-specific bias) cannot be completely eliminated, we have taken reasonable steps to mitigate their impact and documented these limitations for transparency.

Future improvements should focus on expanding to diverse technology ecosystems, increasing dataset size through additional repository collection, and conducting cross-domain validation studies to better understand model generalizability limits.

References -

J. Demšar. Statistical comparisons of classifiers over multiple data sets. The Journal of Machine Learning Research, 7:1–30, 2006. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.

Changes in Thesis, as per examiners comments -

Comment	Section	Thesis updated in response to improvement feedback
Included screenshots of the model selection and execution interfaces for SDPTool	Please refer Chapter 6, Section 6.3.	Various User interface related images, Fig 6.3 to 6.9 added/updated.
Dataset summary table lacks an example row.	Please refer Chapter 3, Section 3.3	Fig. 3.2. Sample Defect Row MLDD. Fig. 3.3. Sample Defect Row MCDD.
Provide an SDPTool working flowchart.	Please refer Chapter 6, Section 6.3.	Fig. 6.2. User Interaction Flow
Computation environment is not clearly mentioned.	Please refer Chapter 4, SubSection 4.5.2	Table 4.3 Device Specifications
Briefly introduce or reference the concepts of Nature-Inspired Optimization (GA, PSO, GWO) to provide clearer	Please refer Chapter 2, Subsection 2.7.1,2.7.2,2.7.3	Section 2.7 Nature-Inspired Optimization Algorithms (GA, PSO, GWO)

context for their application in Chapter 4.		
The thesis lacks statistical significance testing to validate model improvements.	Please refer Chapter 5, Section 5.6 for Statistical Significance Testing	5.6.1 Statistical Significance Testing - Unbalanced Data 5.6.2 Statistical Significance Testing - Balanced Data
Clearly articulate the future scope as moving toward a single, unified/hybrid AI pipeline that performs both multilabel (defect nature) and multiclass (module prediction) inference simultaneously to achieve a complete automated triage system.	Please refer Chapter -7, Section 7.4 Future scope	Updated section with AI pipeline for Unified model. See Fig. 7.1. Future-Scope : Unified Hybrid Model.
SDPTool has not been evaluated through user studies or compared with existing tools.	Please refer Chapter - 6, Section 6.2 Overview of SDPTool workflow	Table 6.1 Comparison of Existing Tools/Frameworks with SDPTool
Justify the dataset sizes by emphasizing that they represent the entire, high-quality, manually-labeled population available from the source repository, rather than a sample.	Please refer Chapter -3 , Section 3.3 Overview of proposed datasets	Please see "Dataset Size Justification" - clearly reflects the entire set of high-quality, defect reports from the respective Spring repositories
Strengthen the discussion on Threats to Validity by explicitly outlining how Cross-Project Defect Prediction (CPDP) could be a next step to validate the model's generalizability across different projects/domains.	Please refer Chapter-4 , Section -4.7.3 Threats to validity	Threats to validity updated to showcase the importance of CPDP to enhance - Domain shift, Vocabulary and terminology differences, Label distribution shift, Contextual generalization

Ensure the unique contribution of the NNLS-based sampling technique is distinctly highlighted in the Discussion as a key methodological innovation.	Please refer Chapter 5, Section - 5.7.1 NNLS-Based Sampling: A Key Methodological Innovation	Showcased the effectiveness of NNLS exclusively for multilabel class balancing.
Augment the discussion with a hypothetical or actual industrial case study to illustrate the seamless integration of SDPTool into a DevOps/CI/CD workflow, maximizing the perception of its practical value.	Please refer Chapter 6 , Section 6.7 Industrial Integration: SDPTool in DevOps/CI/CD Workflows	It clearly demonstrates SDPTool's practical value beyond academic research.
Ensure the Research Objectives (Section 1.3.1) are explicitly phrased as concrete, measurable goals (e.g., "To develop a high-performing CNN-based model for multilabel defect classification").	Please refer Chapter 1, Section 1.3.1 Research objectives	Research objectives are updated to have measurable concrete goals.
Explainability aspects for black-box models like BERT and CNN are minimal, and deeper error analysis of misclassifications is missing.	Please refer Chapter -5 Sections - 5.9.4 Addressing the black-box nature through statistical validation & Section 5.9.5 Future work	Added comprehensive subsections on black-box model interpretability, demonstrating confidence through rigorous statistical validation (10-fold CV, detailed error analysis) while outlining future enhancements including exploring feature ablation.
Extend SDPTool to support multi-language software projects beyond Java (including Python, JavaScript, C++, etc.).	Please refer Chapter 7, Section 7.4 Future scope	Clearly mentioned that Expanding SDPTool to handle multilingual projects would provide several key advantages.
Add more real-world examples in the results and discussion section.	Please Refer Chapter 4, Section 4.7 Discussion Refer Chapter 5, Section 5.7	Paragraph for Real-World Application Example for both Multiclass and Multilabel scenarios.

Fix minor grammatical issues across chapters.	4.6.3 Confusion–matrix insights 6.7.5 Cost-benefit and extensibility 4.7 Discussion	Minor Grammer Issues.